SUMMARY: NUPMY-II

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand operations on arrays.
- Understand the statistical operations on arrays
- Understand the difference between list and array
- Understand the other ways of creating NumPy Arrays

KEY POINTS AND EXAMPLES:

Understand operations on arrays

Transpose

Transposing an array turns its rows into columns and columns into rows just like matrices in mathematics.
In NumPy, we can use the transpose method or the T attribute of the array object to transpose the array.

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

print(arr.transpose())
print("Original array after applying transpose:\n", arr)
```

The output will be:

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
Original array after applying transpose:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Note: The original array remains unchanged after applying the transpose method. The transpose method creates a new array with the transposed elements, but it does not modify the

original array.

The T attribute is a shorthand way to achieve the same result:

```
print(arr.T)
```

The output will be:

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

Sorting

NumPy provides several functions and methods for sorting arrays. Sorting is a common operation in data analysis and numerical computing, and NumPy makes it easy to sort arrays in different ways.

Sorting is to arrange the elements of an array in hierarchical order either ascending or descending. By default, numpy does sorting in ascending order

1. **np.sort() Function**
    - Syntax: np.sort(array, axis=-1)
    - Description: Returns a sorted copy of an array along a specified axis.
    - Parameters:
    - array: The input array to be sorted.
    - axis (optional): Axis along which to sort the array. Default is -1, which sorts along the last axis.

Axis 0 (Rows): This refers to the direction along the rows. Operations along this axis will operate across the columns.
Axis 1 (Columns): This refers to the direction along the columns. Operations along this axis will operate across the rows.

Axis 0 runs vertically down the columns.
Axis 1 runs horizontally across the rows.
For a 2D array, axis=-1 is equivalent to axis=1 since the last axis in a 2D array is the columns.

```
import numpy as np
```

```
# Example 1: Sorting a 1D array
arr1 = np.array([3, 1, 5, 2, 4])
sorted_arr1 = np.sort(arr1)
print("Sorted 1D Array:", sorted_arr1)

# Example 2: Sorting a 2D array along rows (axis=1)
arr2 = np.array([[3, 2, 1],
                 [6, 5, 4]])
sorted_arr2 = np.sort(arr2, axis=1)
print("Sorted 2D Array along rows:")
print(sorted_arr2)
```

The output will be:

```
Sorted 1D Array: [1 2 3 4 5]
Sorted 2D Array along rows:
[[1 2 3]
 [4 5 6]]
```

The np.sort(arr1) function sorts the 1D array arr1 in ascending order.
The np.sort(arr2, axis=1) function sorts the 2D array arr2 along rows, meaning it sorts each row individually in ascending order.
In the example, the first row [3, 2, 1] becomes [1, 2, 3], and the second row [6, 5, 4] becomes [4, 5, 6].

```
import numpy as np

# Example 2D array
arr = np.array([[5, 2, 3],
                [8, 1, 6],
                [7, 4, 9]])

# Sorting along axis 0 (columns)
sorted_arr_axis0 = np.sort(arr, axis=0)
print("Sorted 2D Array along axis 0 (columns):")
print(sorted_arr_axis0)

# Sorting along axis 1 (rows)
sorted_arr_axis1 = np.sort(arr, axis=1)
print("Sorted 2D Array along axis 1 (rows):")
print(sorted_arr_axis1)
```

```
# Sorting along axis -1 (last axis, rows for 2D array)
sorted_arr_axis_minus1 = np.sort(arr, axis=-1)
print("Sorted 2D Array along axis -1 (last axis, rows for 2D
array):")
print(sorted_arr_axis_minus1)
```

The output will be:

```
Sorted 2D Array along axis 0 (columns):
[[5 1 3]
 [7 2 6]
 [8 4 9]]
Sorted 2D Array along axis 1 (rows):
[[2 3 5]
 [1 6 8]
 [4 7 9]]
Sorted 2D Array along axis -1 (last axis, rows for 2D array):
[[2 3 5]
 [1 6 8]
 [4 7 9]]
```

Note: The original array is not modified by np.sort; it returns a sorted copy of the array. To sort the array in place, you can use the sort method on the array itself (e.g., arr1.sort()).

2. **arr.sort(axis=-1) Method**
   The arr.sort() method sorts an array in-place, meaning it modifies the original array rather than creating a new sorted array.

```
import numpy as np

# Create a 2D NumPy array
arr = np.array([[5, 2, 3],
                [8, 1, 6],
                [7, 4, 9]])

# Sort the array in-place along the last axis (axis=-1)
arr.sort(axis=-1)
print("Sorted 2D Array along axis -1 (last axis, which is
columns):")
print(arr)
```

The output will be:

```
Sorted 2D Array along axis -1 (last axis, which is columns):
[[2 3 5]
 [1 6 8]
 [4 7 9]]
```

3. **np.argsort(arr, axis=-1) Function**
   It returns the indices that would sort an array. This can be useful for indirectly sorting arrays or for obtaining the order of elements.

```
import numpy as np

# Create a 1D NumPy array
arr1 = np.array([3, 1, 5, 2, 4])

# Get the indices that would sort the array
sorted_indices1 = np.argsort(arr1)
print("Indices that would sort the 1D array:", sorted_indices1)
```

The output will be:

```
Indices that would sort the 1D array: [1 3 0 4 2]
```

```
import numpy as np

# Create a 2D NumPy array
arr2 = np.array([[3, 2, 1],
                 [6, 5, 4]])

# Get the indices that would sort each row of the array
sorted_indices2 = np.argsort(arr2, axis=1)
print("Indices that would sort each row of the 2D array:")
print(sorted_indices2)
```

The output will be:

```
Indices that would sort each row of the 2D array:
[[2 1 0]
```

```
 [2 1 0]]
```

```
import numpy as np

# Create a 2D NumPy array
arr3 = np.array([[3, 2, 1],
                 [6, 5, 4]])

# Get the indices that would sort each column of the array
sorted_indices3 = np.argsort(arr3, axis=0)
print("Indices that would sort each column of the 2D array:")
print(sorted_indices3)
```

The output will be:

```
Indices that would sort each column of the 2D array:
[[0 0 0]
 [1 1 1]]
```

- When axis=0, np.argsort() sorts each column independently.
- For the first column [3, 6], the sorted order is [3, 6]. The original indices are [0, 1], so the sorted indices for this column are [0, 1].
- For the second column [2, 5], the sorted order is [2, 5]. The original indices are [0, 1], so the sorted indices for this column are [0, 1].
- For the third column [1, 4], the sorted order is [1, 4]. The original indices are [0, 1], so the sorted indices for this column are [0, 1].

Concatenating Arrays

Concatenating arrays in NumPy involves combining two or more arrays into a single array. This can be done along different axes (dimensions). The primary function used for this purpose is np.concatenate.

The np.concatenate function joins arrays along an existing axis.

Syntax:

```
np.concatenate((array1, array2, ...), axis=0)
```

array1, array2, ...: The arrays to be concatenated. They must have the same shape, except in the dimension corresponding to axis.

For example, if you are concatenating along axis=0 (rows), the arrays must have the same number of columns (same shape in all dimensions except the first one). If you are concatenating along axis=1 (columns), the arrays must have the same number of rows.

When concatenating 1D arrays, they simply need to be the same type of array:

```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

result = np.concatenate((arr1, arr2))
print(result)
```

The output will be:

```
[1 2 3 4 5 6]
```

When concatenating 2D arrays along rows (axis=0), the number of columns must be the same:

```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])

result = np.concatenate((arr1, arr2), axis=0)
print(result)
```

The output will be:

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

When concatenating along columns (axis=1), the number of rows must be the same:

```
result = np.concatenate((arr1, arr2), axis=1)
print(result)
```

The output will be:

```
[[1 2 5 6]
 [3 4 7 8]]
```

```
import numpy as np

# Create a 2D NumPy array with shape (2, 3)
arr1 = np.array([[1, 2, 3],
                 [4, 5, 6]])

# Create a 2D NumPy array with shape (2, 2)
arr2 = np.array([[7, 8],
                 [9, 10]])

# Create a 2D NumPy array with shape (1, 3)
arr3 = np.array([[11, 12, 13]])
```

```
# This is possible because they have the same number of columns
concatenated_axis0 = np.concatenate((arr1, arr3), axis=0)
print("Concatenated along rows (axis=0):")
print(concatenated_axis0)
```

The output will be:

```
Concatenated along rows (axis=0):
[[ 1  2  3]
 [ 4  5  6]
 [11 12 13]]
```

```
# This is possible because they have the same number of rows
concatenated_axis1 = np.concatenate((arr1, arr2), axis=1)
print("Concatenated along columns (axis=1):")
print(concatenated_axis1)
```

The output will be:

```
Concatenated along columns (axis=1):
[[ 1  2  3  7  8]
 [ 4  5  6  9 10]]
```

If you attempt to concatenate arrays with different shapes along an axis where the shapes do not match, NumPy will raise a ValueError.

```
import numpy as np

# Create a 2D NumPy array with shape (2, 3)
arr1 = np.array([[1, 2, 3],
                 [4, 5, 6]])

# Create a 2D NumPy array with shape (2, 2)
arr2 = np.array([[7, 8],
                 [9, 10]])

# Attempt to concatenate arr1 and arr2 along rows (axis=0)
try:
    concatenated_axis0 = np.concatenate((arr1, arr2), axis=0)
    print("Concatenated along rows (axis=0):")
    print(concatenated_axis0)
except ValueError:
    print("ValueError")
```

The output will be;

```
ValueError
```

When concatenating along rows (axis=0), the number of columns (dimension 1) must be the same for all arrays. Here, arr1 has 3 columns and arr2 has 2 columns, which violates this requirement.

Reshaping Arrays

Reshaping arrays in NumPy allows you to change the shape or dimensions of an existing array without changing its data.

Reshape Method: NumPy arrays have a reshape() method that allows you to specify the new shape of the array. The new shape should be compatible with the original shape (i.e., the total number of elements should remain the same).

Order of Elements: By default, elements of the array are read row-wise.

```
import numpy as np
```

```
# Create a 1D array with 12 elements
arr1d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
print("Original 1D Array:")
print(arr1d)

# Reshape the 1D array to a 2D array with 4 rows and 3 columns
arr2d = arr1d.reshape(4, 3)
print("Reshaped 2D Array:")
print(arr2d)
```

The output will be:

```
Original 1D Array:
[ 0  1  2  3  4  5  6  7  8  9 10 11]
Reshaped 2D Array:
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

```
# Reshape the 1D array to a 3D array with shape (2, 3, 2)
arr3d = arr1d.reshape(2, 3, 2)
print("Reshaped 3D Array:")
print(arr3d)
```

The output will be:

```
Reshaped 3D Array:
[[[ 0  1]
  [ 2  3]
  [ 4  5]]

 [[ 6  7]
  [ 8  9]
  [10 11]]]
```

```
# Flatten the 2D array back to 1D
flattened = arr2d.reshape(-1)
print("Flattened 1D Array:")
print(flattened)
```

The output will be:

```
Flattened 1D Array:
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

Use -1 in reshape() to automatically calculate the size of one dimension based on the other dimensions.

The reshape() function in NumPy can give errors in specific cases where the new shape specified is not compatible with the original array's size.

```
import numpy as np

# Create a 1D array with 10 elements
arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
print("Original Array:")
print(arr)

# Try to reshape the array to a shape that doesn't match the
number of elements
try:
    arr_reshaped = arr.reshape(3, 4)  # This will raise an error
    print("Reshaped Array:")
    print(arr_reshaped)
except ValueError:
    print("Error occurred")
```

The output will be:

```
Original Array:
[0 1 2 3 4 5 6 7 8 9]
Error occurred
```

Since the original array has 10 elements and a 3x4 array requires 12 elements, NumPy raises a ValueError because the shapes are incompatible.

Splitting Arrays

We can split an array into two or more subarrays. .Splitting arrays in NumPy can be done using the np.split() function.

```python
import numpy as np

# Create a 1D NumPy array
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print("Original Array:")
print(arr)

# Split the array into 5 equal parts
split_arr = np.split(arr, 5)
print("Split Array into 5 parts:")
print(split_arr)

# Create a 2D NumPy array
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print("Original 2D Array:")
print(arr_2d)

# Split the 2D array along axis 1 (columns)
split_arr_2d = np.split(arr_2d, 3, axis=1)
print("Split 2D Array along columns:")
print(split_arr_2d)

# Split the 2D array along axis 0 (rows)
split_arr_2d_ = np.split(arr_2d, 3, axis=0)
print("Split 2D Array along rows:")
print(split_arr_2d_)
```

The output will be:

```
Original Array:
[ 1  2  3  4  5  6  7  8  9 10]
Split Array into 5 parts:
[array([1, 2]), array([3, 4]), array([5, 6]), array([7, 8]),
array([ 9, 10])]
Original 2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Split 2D Array along columns:
```

```
[array([[1],
       [4],
       [7]]), array([[2],
       [5],
       [8]]), array([[3],
       [6],
       [9]])]
Split 2D Array along rows:
[array([[1, 2, 3]]), array([[4, 5, 6]]), array([[7, 8, 9]])]
```

If you want to remove the word "array" from the output and make it look cleaner

```
import numpy as np

# Create a 1D NumPy array
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print("Original Array:")
print(arr)

# Split the array into 5 equal parts
split_arr = np.split(arr, 5)
print("Split Array into 5 parts:")
for part in split_arr:
    print(part)

# Create a 2D NumPy array
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print("Original 2D Array:")
print(arr_2d)

# Split the 2D array along axis 1 (columns)
split_arr_2d = np.split(arr_2d, 3, axis=1)
print("Split 2D Array along columns:")
for part in split_arr_2d:
    print(part)

# Split the 2D array along axis 0 (rows)
split_arr_2d_ = np.split(arr_2d, 3, axis=0)
print("Split 2D Array along rows:")
for part in split_arr_2d_:
```

```
    print(part)
```

The output will be:

```
Original Array:
[ 1  2  3  4  5  6  7  8  9 10]
Split Array into 5 parts:
[1 2]
[3 4]
[5 6]
[7 8]
[ 9 10]
Original 2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Split 2D Array along columns:
[[1]
 [4]
 [7]]
[[2]
 [5]
 [8]]
[[3]
 [6]
 [9]]
Split 2D Array along rows:
[[1 2 3]]
[[4 5 6]]
[[7 8 9]]
```

Understand the statistical operations on arrays

NumPy provides functions to perform many useful statistical operations on arrays.

1. **Mean:** Calculates the arithmetic mean along a specified axis.

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])
```

```
mean_all = np.mean(arr)  # Mean of all elements
mean_axis0 = np.mean(arr, axis=0)  # Mean along axis 0 (columns)
mean_axis1 = np.mean(arr, axis=1)  # Mean along axis 1 (rows)

print("Mean of all elements:", mean_all)
print("Mean along axis 0 (columns):", mean_axis0)
print("Mean along axis 1 (rows):", mean_axis1)
```

The output will be:

```
Mean of all elements: 3.5
Mean along axis 0 (columns): [2.5 3.5 4.5]
Mean along axis 1 (rows): [2. 5.]
```

2. **Median:** Computes the median along a specified axis.

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

median_all = np.median(arr)  # Median of all elements
median_axis0 = np.median(arr, axis=0)  # Median along axis 0
(columns)
median_axis1 = np.median(arr, axis=1)  # Median along axis 1
(rows)

print("Median of all elements:", median_all)
print("Median along axis 0 (columns):", median_axis0)
print("Median along axis 1 (rows):", median_axis1)
```

The output will be:

```
Median of all elements: 3.5
Median along axis 0 (columns): [2.5 3.5 4.5]
Median along axis 1 (rows): [2. 5.]
```

3. **Standard Deviation:** Calculates the standard deviation along a specified axis.

```
import numpy as np
```

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

std_all = np.std(arr)  # Standard deviation of all elements
std_axis0 = np.std(arr, axis=0)  # Standard deviation along axis 0
(columns)
std_axis1 = np.std(arr, axis=1)  # Standard deviation along axis 1
(rows)

print("Standard deviation of all elements:", std_all)
print("Standard deviation along axis 0 (columns):", std_axis0)
print("Standard deviation along axis 1 (rows):", std_axis1)
```

The output will be:

```
Standard deviation of all elements: 1.707825127659933
Standard deviation along axis 0 (columns): [1.5 1.5 1.5]
Standard deviation along axis 1 (rows): [0.81649658 0.81649658]
```

4. **Min and Max:** Computes the minimum and maximum values along a specified axis.

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

min_all = np.min(arr)  # Minimum value of all elements
min_axis0 = np.min(arr, axis=0)  # Minimum value along axis 0
(columns)
min_axis1 = np.min(arr, axis=1)  # Minimum value along axis 1
(rows)

max_all = np.max(arr)  # Maximum value of all elements
max_axis0 = np.max(arr, axis=0)  # Maximum value along axis 0
(columns)
max_axis1 = np.max(arr, axis=1)  # Maximum value along axis 1
(rows)

print("Minimum value of all elements:", min_all)
print("Minimum value along axis 0 (columns):", min_axis0)
print("Minimum value along axis 1 (rows):", min_axis1)
```

```
print("Maximum value of all elements:", max_all)
print("Maximum value along axis 0 (columns):", max_axis0)
print("Maximum value along axis 1 (rows):", max_axis1)
```

The output will be:

```
Minimum value of all elements: 1
Minimum value along axis 0 (columns): [1 2 3]
Minimum value along axis 1 (rows): [1 4]
Maximum value of all elements: 6
Maximum value along axis 0 (columns): [4 5 6]
Maximum value along axis 1 (rows): [3 6]
```

5. **Sum:** Computes the sum of array elements along a specified axis.

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

sum_all = np.sum(arr)  # Sum of all elements
sum_axis0 = np.sum(arr, axis=0)  # Sum along axis 0 (columns)
sum_axis1 = np.sum(arr, axis=1)  # Sum along axis 1 (rows)

print("Sum of all elements:", sum_all)
print("Sum along axis 0 (columns):", sum_axis0)
print("Sum along axis 1 (rows):", sum_axis1)
```

The output will be:

```
Sum of all elements: 21
Sum along axis 0 (columns): [5 7 9]
Sum along axis 1 (rows): [ 6 15]
```

Understand the difference between list and array

Feature	Python List	NumPy Array
Data Type	Can contain mixed data types	Contains elements of the same data type

Element-wise Operations	Lists do not support element wise operations, for example, addition, multiplication, etc. because elements may not be of same type. Requires loops or list comprehensions	Arrays support element wise operations. For example, if A1 is an array, it is possible to say A1/3 to divide each element of the array by 3.
Mathematical Functions	Limited, requires manual implementation	Extensive mathematical functions available
Usage	General-purpose, flexible	Specifically for numerical and scientific computing
Library Support	Built-in to Python List is a part of core Python.	Requires importing the NumPy library. Array (ndarray) is a part of NumPy library.

Understand the other ways of creating NumPy Arrays

a. **np.zeros(shape, dtype=float):** Creates a new array filled with zeros. The shape can be an integer (for a 1D array) or a tuple of integers (for multidimensional arrays). By default, the data type of the array created by zeros() is float.

```
import numpy as np

# 1D array of 5 zeros
print("1D array of 5 zeros:")
a = np.zeros(5)
print(a)

# 2D array of 3x4 zeros
print("2D array of 3x4 zeros:")
b = np.zeros((3, 4))
print(b)

# 3D array of 2x3x4 zeros
print("3D array of 2x3x4 zeros:")
c = np.zeros((2, 3, 4))
print(c)
```

The output will be:

```
1D array of 5 zeros:
[0. 0. 0. 0. 0.]
2D array of 3x4 zeros:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
3D array of 2x3x4 zeros:
[[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
```

By default, np.zeros() creates arrays of float data type, but you can specify a different data type using the dtype parameter (dtype=int, for example).

b. **np.ones(shape, dtype=float):** Creates a new array filled with ones, with the specified shape.By default, the data type of the array created by ones() is float.

```
import numpy as np

# 1D array of 3 ones
c = np.ones(3)
print("1D array of 3 ones:")
print(c)


# 2D array of 2x3 ones with integer data type
d = np.ones((2, 3), dtype=int)
print("2D array of 2x3 ones (integer data type):")
print(d)
```

The output will be:

```
1D array of 3 ones:
[1. 1. 1.]
2D array of 2x3 ones (integer data type):
[[1 1 1]
```

```
 [1 1 1]]
```

c. **np.full(shape, fill_value):** Creates a new array filled with the specified value.The default data type for np.full() is determined by the data type of the fill_value provided. If fill_value is an integer, the default data type will be an integer. If fill_value is a float, the default data type will be a float.

```
import numpy as np

# 1D array of length 5 filled with the value 7
array_1d = np.full(5, 7)
print("1D array filled with 7:")
print(array_1d)

# 2D array of shape 3x4 filled with the value -1
array_2d = np.full((3, 4), -1)
print("2D array filled with -1:")
print(array_2d)

# 3D array of shape 2x3x4 filled with the value 3.14
array_3d = np.full((2, 3, 4), 3.14)
print("3D array filled with 3.14:")
print(array_3d)
```

The output will be:

```
1D array filled with 7:
[7 7 7 7 7]
2D array filled with -1:
[[-1 -1 -1 -1]
 [-1 -1 -1 -1]
 [-1 -1 -1 -1]]
3D array filled with 3.14:
[[[3.14 3.14 3.14 3.14]
  [3.14 3.14 3.14 3.14]
  [3.14 3.14 3.14 3.14]]

 [[3.14 3.14 3.14 3.14]
  [3.14 3.14 3.14 3.14]
  [3.14 3.14 3.14 3.14]]]
```

d. **np.eye(N, M, k):** Creates an NxM identity matrix with ones on the k-th diagonal. The np.eye() function in NumPy creates a 2-dimensional array (matrix) where:
   - The diagonal elements are ones (1).
   - All other elements are zeros (0).

   Parameters:
   - N: Number of rows in the output matrix.
   - M: Number of columns in the output matrix.
   - k: Index of the diagonal where ones are placed. Zero-indexed. k=0 is the main diagonal, k>0 is above the main diagonal, and k<0 is below the main diagonal.
   - dtype: Optional data type of the output array. Defaults to float.

   np.eye(N) creates an N x N identity matrix (2D array).
   np.eye(N, M) creates an N x M matrix with ones on the main diagonal (k=0).
   np.eye(N, M, k) creates an N x M matrix with ones on the k-th diagonal.

```
import numpy as np

# 3x3 identity matrix (N=3, M=None, k=0)
identity_3x3 = np.eye(3)
print("3x3 Identity Matrix:")
print(identity_3x3)

# 3x4 identity-like matrix (N=3, M=4, k=0)
identity_3x4 = np.eye(3, 4)
print("3x4 Identity-Like Matrix:")
print(identity_3x4)

# 4x4 identity matrix with ones on the first upper diagonal (N=4,
M=4, k=1)
identity_shifted = np.eye(4, k=1)
print("4x4 Identity Matrix with Shifted Diagonal:")
print(identity_shifted)
# Create a 5x5 identity-like matrix with ones on the 2nd diagonal
below the main diagonal (k=-2)
identity_k2 = np.eye(5, k=-2)
print("5x5 Identity-Like Matrix with k=-2:")
print(identity_k2)
```

The output will be:

```
3x3 Identity Matrix:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
3x4 Identity-Like Matrix:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]]
4x4 Identity Matrix with Shifted Diagonal:
[[0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]
 [0. 0. 0. 0.]]
5x5 Identity-Like Matrix with k=-2:
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]]
```

Note: np.eye() does not create 1-dimensional (1D) arrays or 3-dimensional (3D) arrays directly. It specifically creates 2D arrays/matrices.

e. **np.arange(start, stop, step):** The np.arange() function in NumPy creates a new 1-dimensional array (ndarray) with evenly spaced values within a specified range. It does not directly create 2-dimensional (2D) or 3-dimensional (3D) arrays.

   Parameters:
   - start: Start of the interval (inclusive). If only one argument is provided, this is the stop value, and start defaults to 0.
   - stop: End of the interval (exclusive).
   - step: Spacing between values. Defaults to 1. If specified as a float, the results are unpredictable.

   The np.arange() function in NumPy is analogous to Python's built-in range() function. Both np.arange() and range() are used to generate sequences of numbers with similar syntax and functionality, but np.arange() produces NumPy arrays, which are more versatile for numerical computations.

```
import numpy as np

# Create an array with values from 0 to 9 (exclusive)
```

```
array1 = np.arange(10)
print("Array with default start and step:")
print(array1)

# Create an array with values from 3 to 12 (exclusive)
array2 = np.arange(3, 13)
print("Array with specified start and default step:")
print(array2)

# Create an array with values from 0 to 10 with a step of 2
array3 = np.arange(0, 11, 2)
print("Array with specified start, stop, and step:")
print(array3)
```

The output will be:

```
Array with default start and step:
[0 1 2 3 4 5 6 7 8 9]
Array with specified start and default step:
[ 3  4  5  6  7  8  9 10 11 12]
Array with specified start, stop, and step:
[ 0  2  4  6  8 10]
```

f. **np.random.random(shape):** Creates a new array(ndarray) with random float values uniformly distributed between 0 (inclusive) and 1 (exclusive), with a specified shape.

   Parameters:
   - size: The shape of the output array. This can be a single integer to specify the length of a 1D array, or a tuple of integers to specify the shape of a multidimensional array.

```
import numpy as np

# Create a 1D array with 5 random float values between 0 and 1
array_1d = np.random.random(5)
print("1D Array:")
print(array_1d)

# Create a 2D array with shape (2, 3) containing random float
values
```

```
array_2d = np.random.random((2, 3))
print("2D Array:")
print(array_2d)
```

The output will be:

```
1D Array:
[0.0314099  0.53892888 0.99910604 0.05308781 0.28354151]
2D Array:
[[0.989234   0.35481144 0.29489286]
 [0.32336212 0.23471922 0.92003697]]
```

g. **np.linspace(start, stop, num, endpoint, restep, dtype):** Creates a new array with num evenly spaced values within the specified range. This function is particularly useful when you need a precise number of points between a start and stop value, inclusive of both endpoints.

   Parameters:
   - start: The starting value of the sequence.
   - stop: The end value of the sequence.
   - num: The number of samples to generate. Default is 50.
   - endpoint: If True (default), stop is the last value. If False, stop is not included.
   - retstep: If True, return (samples, step), where step is the spacing between samples.
   - dtype: The data type of the output array. If not specified, it is inferred from other input arguments.

```
import numpy as np

# Create an array with 5 evenly spaced values from 0 to 1
(inclusive)
array1 = np.linspace(0, 1, 5)
print("Array with 5 evenly spaced values from 0 to 1:")
print(array1)

# Create an array with 10 values from 0 to 5 (inclusive)
array2 = np.linspace(0, 5, 10)
print("Array with 10 evenly spaced values from 0 to 5:")
print(array2)
```

The output will be:

```
Array with 5 evenly spaced values from 0 to 1:
[0.   0.25 0.5  0.75 1.  ]
Array with 10 evenly spaced values from 0 to 5:
[0.         0.55555556 1.11111111 1.66666667 2.22222222 2.77777778
 3.33333333 3.88888889 4.44444444 5.        ]
```